

Xây dựng nền tảng chia sẻ video trực tuyến với hệ thống chuyển mã HLS tự động trên kiến trúc Serverless Container và Event-Driven

Báo cáo tiến độ tuần 9 - 03/10/2026 đến 09/10/2026

Đề tài	Xây dựng nền tảng chia sẻ video trực tuyến với hệ thống chuyển mã HLS tự động trên kiến trúc Serverless Container và Event-Driven
Sinh viên thực hiện	Đình Quốc Cường (523H0008) Võ Huỳnh Minh Đức (523H0014)
Giảng viên hướng dẫn	ThS. Mai Văn Mạnh
Tuần báo cáo	Tuần 9 (03/10/2026 - 09/10/2026)
Tỷ lệ hoàn thành tuần	100%

1. Đối chiếu kế hoạch và kết quả

Tuần 9 đóng gói bộ chuyển mã thành một ảnh Docker. Đây là mắt xích biến FFmpeg từ một công cụ chạy trên máy cá nhân thành một đơn vị tính toán mà hạ tầng có thể tạo ra hàng loạt rồi hủy đi. Hai quyết định của tuần này đều xuất phát từ ràng buộc thật chứ không phải sở thích: chọn ảnh nền nào, và tách quá trình dựng làm mấy tầng.

Kế hoạch	Kết quả	Trạng thái
Đóng gói bộ chuyển mã FFmpeg bằng Docker Multi-stage Container.	Đã hoàn thành tại transcoder/Dockerfile với hai tầng : tầng builder cài phụ thuộc, tầng runtime chỉ nhận kết quả đã dựng. Nhờ vậy công cụ dựng không đi theo vào ảnh cuối, giảm cả dung lượng lẫn diện tích bị tấn công.	Hoàn thành
Cài FFmpeg vào ảnh container.	Đã hoàn thành. Ảnh nền là node:24-slim chứ không phải bản Alpine, vì FFmpeg cần glibc còn Alpine dùng musl. Chọn Alpine cho nhỏ rồi phải tự biên dịch lại FFmpeg là đánh đổi sai hướng cho đồ án này.	Hoàn thành
Kiểm chứng container sinh đủ ba mức chất lượng và ảnh đại diện.	Đã hoàn thành. Một lần gọi FFmpeg sinh cả ba mức 360p, 720p và 1080p , đọc tệp nguồn đúng một lần rồi nạp cho ba chuỗi xử lý song song. Ảnh đại diện trích ở một lượt riêng. Sau đó tiến trình tự ghép master playlist từ kết quả thật.	Hoàn thành
Rà soát quyền chạy của tiến trình trong container.	Đã hoàn thành. Container chạy dưới người dùng appuser chứ không phải root, nên nếu có lỗ hổng trong FFmpeg bị khai thác qua một tệp video độc hại thì phạm vi thiệt hại bị giới hạn.	Hoàn thành

2. Kết quả kỹ thuật chính

- **Một lần gọi FFmpeg cho cả ba mức chất lượng.** Gọi ba lần riêng sẽ giải mã tệp nguồn ba lần, mà giải mã là phần tốn nhất. Đọc một lần rồi nạp cho ba chuỗi mã hoá tiết kiệm đúng phần đó.
- **Ảnh nền chọn theo ràng buộc thư viện, không theo dung lượng.** FFmpeg cần glibc; Alpine dùng musl. Đây là lý do kỹ thuật cụ thể khiến ảnh lớn hơn bản Alpine, và báo cáo nêu rõ thay vì để người đọc tự đoán.

- **Dựng nhiều tầng giữ công cụ dựng ra khỏi ảnh chạy.** Trình biên dịch và các gói chỉ cần lúc cài đặt không có mặt trong ảnh cuối, nên ảnh vừa nhẹ hơn vừa ít thứ để khai thác hơn.
- **Chạy dưới người dùng không đặc quyền.** Bộ chuyển mã xử lý tệp do người lạ tải lên — đúng loại đầu vào không tin được. Chạy bằng root là cho một tệp video độc hại toàn quyền trong container.
- **Master playlist do tiến trình tự ghép sau khi mã hoá xong,** không phải do FFmpeg sinh, nên nó ghi được các giá trị đo từ chính sản phẩm thật thay vì chép lại tham số cấu hình.

3. Minh chứng và kết quả kiểm tra

Minh chứng	Nội dung
transcoder/Dockerfile	Hai tầng builder và runtime, cài FFmpeg, chạy dưới appuser.
transcoder/src/transcoder.js	Một lần gọi FFmpeg cho ba mức, trích ảnh đại diện, ghép master playlist.
transcoder/src/config.js	Tham số ba mức chất lượng và thời lượng mỗi đoạn.
transcoder/src/index.js	Điểm vào của container, ba chế độ chạy batch, worker và manual.

Lệnh kiểm tra	Kết quả
<code>docker build -t transcoder transcoder/</code>	Dựng thành công qua cả hai tầng
<code>docker run transcoder ffmpeg -version</code>	FFmpeg có trong ảnh và chạy được
<code>docker run transcoder id</code>	uid không phải 0 — tiến trình chạy dưới appuser
Chạy thử trên một video mẫu	Sinh đủ ba thư mục 360p, 720p, 1080p kèm master.m3u8 và thumbnail.jpg
Đếm số đoạn của mỗi mức	Ba mức có số đoạn bằng nhau — các mức cắt trùng biên

4. Giới hạn và rủi ro hiện tại

- Ảnh container lớn hơn đáng kể so với phương án nền Alpine, do phải dùng bản có glibc để chạy FFmpeg. Đây là đánh đổi đã cân nhắc chứ không phải thiếu sót.
- Phiên bản FFmpeg lấy theo kho gói của bản phân phối nên không ghim được chính xác; nâng cấp ảnh nền có thể kéo theo đổi phiên bản FFmpeg mà không chủ động.
- Container chưa có kiểm tra sức khỏe, nhưng đây là công việc chạy một lần rồi kết thúc chứ không phải dịch vụ thường trực, nên khái niệm đó không áp dụng.
- Mỗi video do **một container duy nhất** xử lý, không chia nhỏ theo khối, nên thời gian xử lý tăng tuyến tính theo thời lượng video và không rút ngắn được bằng cách thêm tài nguyên.

5. Cam kết cho tuần tiếp theo

	Nội dung
1	Xây dựng luồng bất đồng bộ Amazon S3 Event Notification đến Amazon SQS.
2	Kết nối Amazon SQS với hàm Lambda nộp việc và AWS Batch trên Fargate.
3	Thiết lập hàng đợi thư chết để giữ lại công việc thất bại thay vì đánh mất.